

I. BASE DE DONNEES – PARTIE C

Les REQUETES : INTERROGATION de la BASE de DONNEES

- SQL : mono-table

INTERROGATION de la BASE de DONNEES :

- on effectue des requêtes, on parle de requêtes simples (mono) et requêtes complexes (multi)

Syntaxe d'une requête SQL

SELECT on a ce que l'on veut obtenir comme resultat, on ajoute les proprietes qui afficheront le resultat

FROM où est ce que on va chercher les informations, dans quelleS tableS?

WHERE on va ajouter des conditions, des spécificités

SELECT :

- propriété
- **DISTINCT** : **SELECT DISTINCT** supprimer les duplicatas
- on peut ajouter des fonctions

FROM :

- on fait apparaitre les tables dans lesquelles on va aller chercher l'information (multi : procedurale, relationnelle, inner)

WHERE :

- on utilise des predicats, cad on effectue des restrictions de lignes, des filtres horizontaux, comparaisons.....
- on utilise des fonctions **ASC pour croissant**
- **ORDER BY** l'objectif est d'utiliser un classement croissant ou décroissant (**DESC**)
Ce **DESC** n'est pas lié au **DESC** de description de la table

GROUP BY : on va construire un regroupement, un ensemble

HAVING : sur ce sous ensemble on ne recupere que certaines colonnes

Fonction SQL:

-fonction numériques

COS, SIN, TAN, TRUNC, SQRT.....

-fonction chaînes de caractères

UPPER tt en majuscule

SUBSTR extraire des sous-chaînes

CONCAT pour concatener (|| oracle, CONCAT(.....)MySQL)

-fonctions pour les dates

TRUNC tronquer des dates

SYSDATE date courante

ROUND arrondir une date

-fonctions sur les agrégats

AVG moyenne

COUNT compter le nombre de lignes

MAX maximum

MIN minimum

SUM somme

VARIANCE variance

STDDEV ecart type

- SQL : multi-tables

Les REQUETES :

```
SELECT  
FROM  
WHERE
```

En mono table on ne travaille que sur une seule table. En multi table on travaille sur **PLUSIEURS** tables.

JOINTURE MULTI TABLES : nous obtenons le même résultat.

- des jointures de la forme relationnelles
- des jointures de la forme procédurales
- des jointures en INNER JOIN

Jointures Relationnelles :

```
SELECT  
FROM => nous n'avons qu'une seule clause FROM  
WHERE
```

- on va utiliser des préfixes et des suffixes. on aura les **préfixes** dans le **SELECT** et dans le **WHERE**. on aura les **suffixes** dans le **FROM**

Format de la requête :

```
SELECT T1.a, T3.b, T3.c  
FROM table1 T1, table2 T2, table3 T3  
WHERE T1.a=T2.b /*liaison entre les tables table1 et table2*/  
AND T3.c = T2.d  
AND T3.c = "XYZ";
```

Exemple :

```
EQUIPE(nume,nom_equipe)  
PROJET(nump,designation,nume#,pbdg)
```

```
Nom des équipes ayant un budget supérieur à 100K.  
SELECT e.nom_equipe /*affichage*/  
FROM equipe e, projet p  
WHERE e.nume = p.nume /*jointure entre equipe et projet*/  
AND p.pbdg > 100 ;
```

JOINTURES PROCEDURALES

Remarques : En SQL toutes les requêtes multi-tables quelque soit leur forme donnent le même résultat.

Les requêtes procédurales sont des requêtes IMBRIQUEES.

Forme de la requête avec le **IN** :

```
SELECT a1, a2, a3.....
```

```
FROM table1
```

```
WHERE PK ou FK
```

ATTENTION

IN

```
SELECT FK OU PK
```

```
FROM table 2
```

```
WHERE condition/predicat.....
```

Remarque : on peut avoir un nombre indéfini de requêtes imbriquées (c'est le cas aussi pour les jointures de la forme relationnelle et aussi pour les INNER.....)

Forme de la requête avec le $\equiv$:

```
SELECT a1, a2, a3.....
FROM table1
WHERE PK ou FK ATTENTION
      =
      SELECT FK OU PK
      FROM table 2
      WHERE condition/predicat.....
```

Différence entre le IN et le $\equiv$

IN : on est dans un ensemble, le SELECT qui se situe après le IN peut restituer plusieurs valeurs

$\equiv$: le SELECT qui se situe après le $\equiv$ doit restituer une seule valeur

Schema Relationnel

EQUIPE (nume, nome)

PROJET (nump, libelle, #nequipe, pbudg)

EMPLOYE (numc, nomc)

AFFECTER (#nume, #numc)

Exemple :

Q1 : Donner le nom de l'équipe qui a un budget superieur à 100K.

```
SELECT nome          /*on affiche le nom de l'équipe de la table equipe*/
FROM equipe          /*on travaille dans la table equipe*/
WHERE nume      /*nous avons une jointure procédurale entre la table equipe via
nume et la table projet via nequipe*/
      IN
      (SELECT nequipe
      FROM projet
      WHERE pbudg > 100000)
;
```

EXEMPLES de REQUETES

Schema Relationnel

EMP (nocemp, nomemp, prenomemp, salaireemp, ageemp)

Agregat :

Q1 : salaire moyen

```
SELECT AVG(salaireemp) –utilisation de la fonction AVG  
FROM EMP
```

Q2 : combien avons nous de salariés ?

```
SELECT COUNT(nocemp)  
FROM EMP
```

Q3 : combien avons nous de salariés de plus de 50 ans ?

```
SELECT COUNT(nocemp)  
FROM EMP
```

WHERE ageemp > 50;

Remarque : “50” signifie que c’est une chaine de caractère alors que 50 est bien un integer

Q4 : Somme des salaires des employés

```
SELECT SUM(salaireemp)  
FROM EMP
```

Schema Relationnel

EQUIPE (nume, nome)

PROJET (nump, libelle, #nequipe, pbudg)

EMPLOYE (numc, nomc)

AFFECTER (#nume, #numc)

Q5 : Afficher le numéro des projets ayant un budget de projet supérieur à 100 000 euros

SELECT nump --affichage du résultat

FROM projet - - on travaille dans la table projet

WHERE pbudg > 100000; - -on doit respecter la contrainte, la restriction du budget supérieur à 100000 euros

Q6 : Afficher le numéro des projets et le nom des projets ayant un budget de projet supérieur à 100 000 euros

SELECT nump, libelle --affichage du résultat

FROM projet - - on travaille dans la table projet

WHERE pbudg > 100000; - -on doit respecter la contrainte, la restriction du budget supérieur à 100000 euros

Q7 : Afficher le nom des projets par ordre alphabétique

SELECT libelle

FROM projet

--- dans ce cas on a la liste des projets avec le libelle

SELECT libelle

FROM projet

ORDER BY 1

--- dans ce cas on a la liste des projets avec le libelle en ordre alphabétique

Q8 : Afficher le nom du projet et le budget dont le budget est maximal

SELECT libelle, MAX(pbudg) - -utilisation d'une fonction MAX

FROM projet

Q9 : Pour chaque équipe, afficher son numéro et compter le nombre de projets de plus de 20 000 euros qui lui sont associés.

SELECT nequipe, COUNT (nump) - - on compte le nombre de projets

FROM projet

WHERE pbudb > 20 000 - -on doit respecter la contrainte d'un budget supérieur à 20 000 euros

GROUP BY nequipe - - on regroupe par nequipe pour avoir par équipe le numéro de chaque projet auquel cette équipe sera affectée avec en plus le budget du projet.

Remarque : **ERREUR**

SELECT nume, count(nump) - - nume est lié à la table équipe

FROM PROJET,EQUIPE - - on travaille sur 2 tables qui sont projet et équipe

WHERE pbudg>20000

Dans ce cas nous sommes sur des requêtes multi tables, complexes et on doit faire un lien (nous faisons des requêtes dans un SGBD Relationnel) entre les PK et FK des tables en questions c'est-à-dire proposer une égalité entre nume de équipe et nequipe de projet

projet.nequipe = equipe.nume (à mettre dans le WHERE)

Test d'Existence : utilisation du EXISTS, du NOT EXISTS

SELECT

FROM

WHERE : on teste un prédicat

L'utilisation de cette classe dans le WHERE permet de valider la valeur ou PAS. Si c'est validé cela signifie qu'il y a au minimum un enregistrement

On l'utilise via des requêtes procedurales.

Exemple :

Schema Relationnel

EQUIPE (**nume**, nome)

PROJET (**nump**, libelle,**#nequipe**,pbudg)

EMPLOYE (numc, nomc)

AFFECTER (#nume, #numc)

L'employé « DUPON » est il affecté à une équipe ?

```
SELECT e.nomc
FROM employe e
WHERE EXISTS
    (
        SELECT a.numc
        FROM affecter a
        WHERE a.numc = e.numc

        AND e.nomc LIKE '%DUPON%'
    );
```

```
SELECT
FROM
WHERE NOT EXISTS
(
    SELECT
    FROM
    WHERE
);
```

Remarque : on peut utiliser le EXISTS lors du CREATE TABLE, cela évite de faire un DROP.

Opérateurs associés aux EXISTS

Utilisation des opérateur : = ; != ; > ; < ; <= ; >= qui sont suivis de ALL ou ANY

Le ALL fait une comparaison qui si elle est VRAIE n'est validée que si c'est pour TOUS LES ELEMENTS

Le ANY, on effectue une comparaison qui est VRAIE si elle est vérifiée pour au moins, au minimum un enregistrement ramené par le SELECT IMBRIQUE

Exemple :

Schema Relationnel

EQUIPE (nume, nome)

PROJET (nump, libelle, #nequipe, pbudg)

EMPLOYE (numc, nomc)

AFFECTER (#nume, #numc)

Donner le projet qui a le plus gros budget :

```
SELECT nump
FROM projet
WHERE pbudg >= ALL
(
    SELECT pbudg FROM projet
);
```

/*on verifie que c'est VRAI pour le budget dans tous les budgets de la table pour pbudg*/

Autre version :

```
SELECT nump
FROM projet
WHERE pbudg =
(
    SELECT MAX(pbudg)
    FROM projet
);
```

/*on fait pareil ici avec la fonction MAX*/

Produit CARTESIEN (Algèbre Relationnelle)

On vient faire une multiplication avec une distribution pour chaque élément.

Schema Relationnel

EQUIPE (nume, nome)

PROJET (nump, libelle, #nequipe, pbudg)

EMPLOYE (numc, nomc)

AFFECTER (#nume, #numc)

Exemple :

Donner toutes les distributions entre les EMPLOYE et les EQUIPES

SELECT e.*, c.*

FROM employe e, equipe c

Resultat

Numc , nomc , nume, nome

UNION (Algèbre Relationnelle)

Il s'agit du regroupement non exclusif de deux ensembles

On utilise cet operateur entre deux tables.

Exemple :

SELECT nume FROM equipe

UNION

SELECT nume FROM affecter

DIFFERENCE (Algèbre Relationnelle)

Utilisation de l'opérateur **MINUS**

Exemple :

Quels sont les numeros d'équipe pour lesquelles aucun employé n'est affecté

/*on doit trouver l'ensemble des nume dans la table qui les a tous cad equipe à laquelle je vais « enlever » ceux qui sont déjà affectés donc on aura ceux non affectés*/

SELECT nume FROM equipe

MINUS

SELECT nume FROM affecter

INTERSECT (Algèbre Relationnelle)

Utilisation de l'opérateur **INTERSECT**

On travaille sur deux ensembles avec comme résultat l'intersection entre les deux.

Exemple :

Quels sont les employés qui sont affectés à la fois à l'équipe 1 et 2 ?

```
SELECT nume FROM affecter WHERE numc = '1'
```

```
INTERSECT
```

```
SELECT nume FROM affecter WHERE numc = '2'
```